



Operating Systems

[1500WETOPS]

José Oramas



Processor Scheduling – Real-Time Algorithms

[Deciding who can use the processor in the very near future]

José Oramas

Processor Scheduling

Long-Term Scheduling

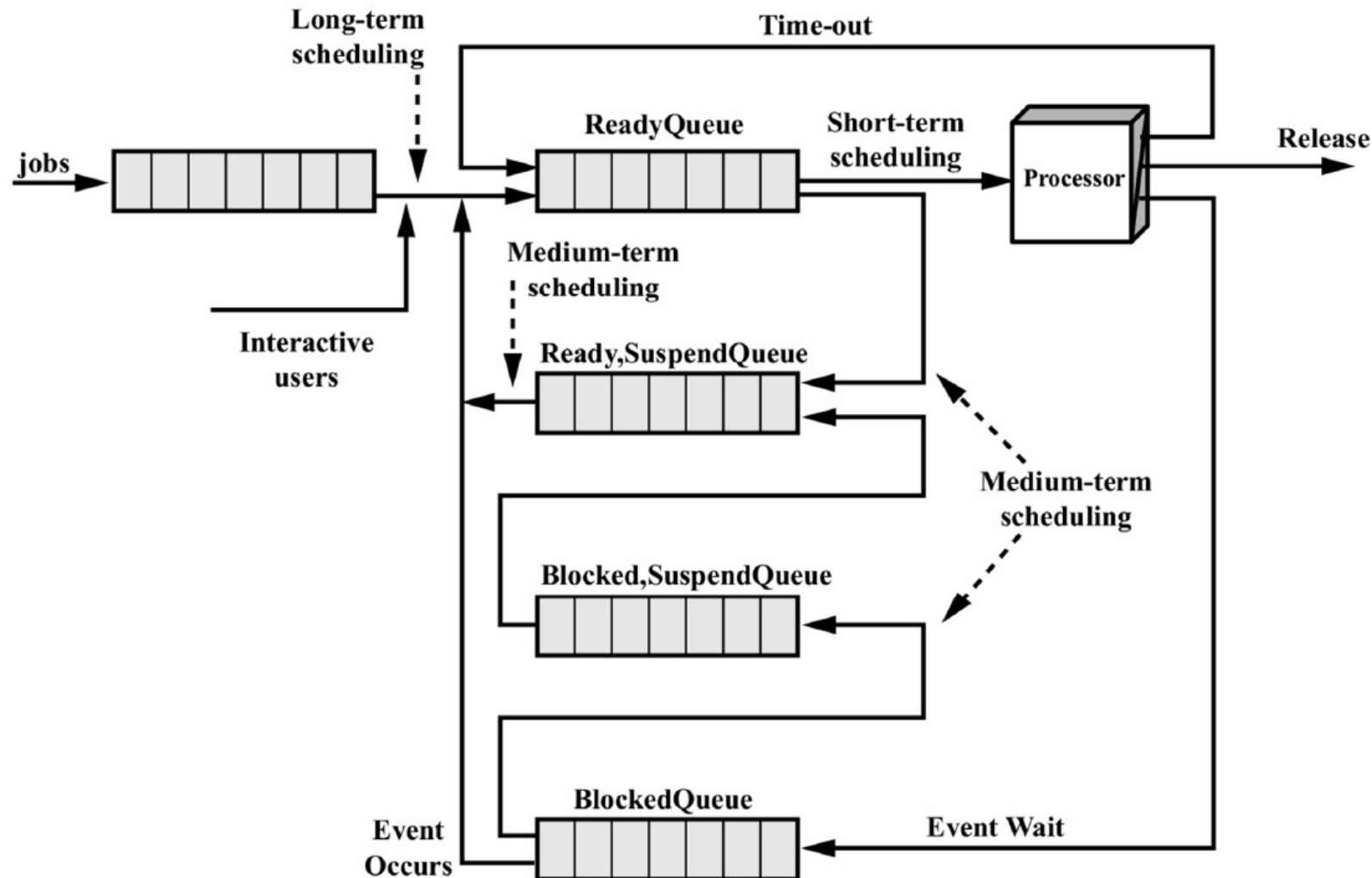
- New → Ready state.
- Decisions:
 - When to activate a process?
 - Which process to activate?

Medium-Term Scheduling

- Processes that move into suspend state

Short-Term Scheduling

- Decides which process can use the processor.



Methodology

Notation: Scheduling Models / Problems

- Universal notation: $\alpha|\beta|\gamma$
 - α : Machine Environment
 - β : Pre-conditions
 - γ : Function to be minimized

Short-Term Scheduling Algorithms

Real-Time Short-Term Scheduling Algorithms

Short-Term Scheduling Algorithms

Real Time Algorithms

- **Problem:** Tasks i have associated deadlines d_i .
[hard vs. soft deadlines]
- Often additional knowledge regarding duration of tasks is available.
- Static (offline) VS. Dynamic (online) variants.
- Types of Static variants:
 - *Table-driven*: create in advanced a table showing the execution order of the tasks. [some tasks may be omitted]
 - *Priority-driven*: output $\rightarrow$ each task is assigned a priority.
[no table is involved]

Real-Time Short-Term Scheduling Algorithms

- **Earliest Deadline First (EDF) – Earliest Due Date (EDD)**
 - AKA. Jackson's Rule
 - Schedule tasks according to ascending deadline
 - Task j has a deadline d_j will minimize
 - $L_{max} = \max_j (C_j - d_j)$
 $\rightarrow 1 \mid \mid L_{max}$

Real-Time Short-Term Scheduling Algorithms

- **Earliest Deadline First (EDF) – Earliest Due Date (EDD)**

- AKA. Jackson's Rule
- Schedule tasks according to ascending deadline
- Task j has a deadline d_j will minimize
 - $L_{max} = \max_j (C_j - d_j)$
 $\rightarrow 1 \mid \mid L_{max}$

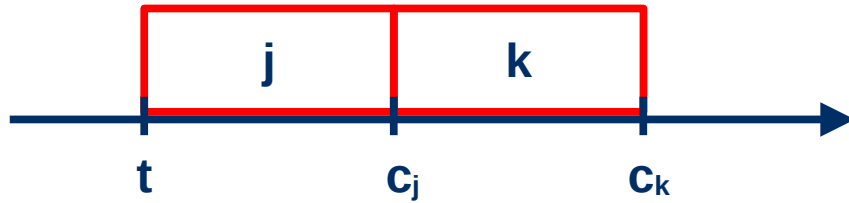
- **Consequently,**

- If deadlines are feasible, i.e. $L_{max} \leq 0$
 $\rightarrow$ The EDF algorithm will meet all of them

Real-Time Short-Term Scheduling Algorithms

- Earliest Deadline First (EDF) – Earliest Due Date (EDD) – Optimality

Consider the ordering S that is optimal but not EDF, then

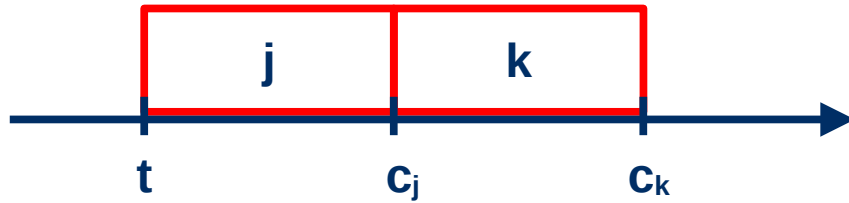


with $d_k < d_j$

Real-Time Short-Term Scheduling Algorithms

- **Earliest Deadline First (EDF) – Earliest Due Date (EDD) – Optimality**

Consider the ordering S that is optimal but not EDF, then



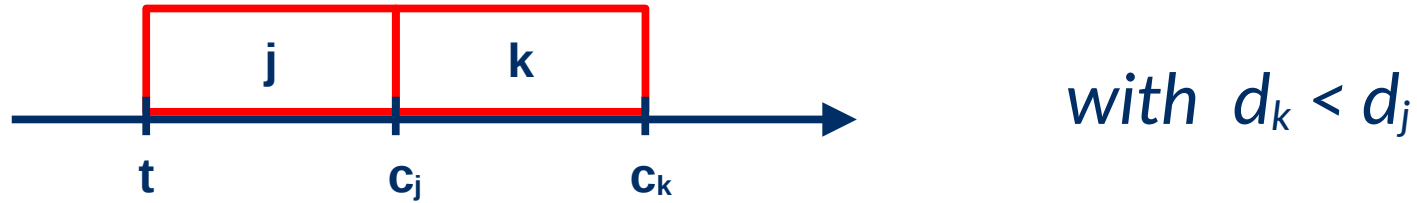
with $d_k < d_j$

Then $\rightarrow C_j = t + p_j$ and $C_k = t + p_j + p_k$

Real-Time Short-Term Scheduling Algorithms

- **Earliest Deadline First (EDF) – Earliest Due Date (EDD) – Optimality**

Consider the ordering S that is optimal but not EDF, then



$$\text{Then } \rightarrow C_j = t + p_j \text{ and } C_k = t + p_j + p_k$$

Let's consider an ordering S' by switching processes k and j

$$\text{Then } \rightarrow C'_k = t + p_k \text{ and } C'_j = t + p_k + p_j$$

Real-Time Short-Term Scheduling Algorithms

- Earliest Deadline First (EDF) – Earliest Due Date (EDD) – Optimality

Considering: $d_k < d_j$

$C_j = t + p_j$	$C'_j = t + p_k + p_j$
$C_k = t + p_j + p_k$	$C'_k = t + p_k$

Real-Time Short-Term Scheduling Algorithms

- **Earliest Deadline First (EDF) – Earliest Due Date (EDD) – Optimality**

Considering: $d_k < d_j$

$C_j = t + p_j$	$C'_j = t + p_k + p_j$
$C_k = t + p_j + p_k$	$C'_k = t + p_k$

Then:

$$C'_j - d_j = C_k - d_j$$

Real-Time Short-Term Scheduling Algorithms

- Earliest Deadline First (EDF) – Earliest Due Date (EDD) – Optimality

Considering: $d_k < d_j$ $C_j = t + p_j$ $C'_j = t + p_k + p_j$
 $C_k = t + p_j + p_k$ $C'_k = t + p_k$

Then:

$$C'_j - d_j = C_k - d_j < C_k - d_k$$

Real-Time Short-Term Scheduling Algorithms

- Earliest Deadline First (EDF) – Earliest Due Date (EDD) – Optimality

Considering: $d_k < d_j$ $C_j = t + p_j$ $C'_j = t + p_k + p_j$
 $C_k = t + p_j + p_k$ $C'_k = t + p_k$

Then:

$$C'_j - d_j = C_k - d_j < C_k - d_k$$

$$C'_k - d_k = t + p_k - d_k$$

Real-Time Short-Term Scheduling Algorithms

- **Earliest Deadline First (EDF) – Earliest Due Date (EDD) – Optimality**

Considering: $d_k < d_j$ $C_j = t + p_j$ $C'_j = t + p_k + p_j$
 $C_k = t + p_j + p_k$ $C'_k = t + p_k$

Then:

$$C'_j - d_j = C_k - d_j < C_k - d_k$$

$$C'_k - d_k = t + p_k - d_k < t + p_k + p_j - d_k$$

Real-Time Short-Term Scheduling Algorithms

- Earliest Deadline First (EDF) – Earliest Due Date (EDD) – Optimality

Considering: $d_k < d_j$ $C_j = t + p_j$ $C'_j = t + p_k + p_j$
 $C_k = t + p_j + p_k$ $C'_k = t + p_k$

Then:

$$C'_j - d_j = C_k - d_j < C_k - d_k$$

$$C'_k - d_k = t + p_k - d_k < t + p_k + p_j - d_k = C_k - d_k$$

Real-Time Short-Term Scheduling Algorithms

- Earliest Deadline First (EDF) – Earliest Due Date (EDD) – Optimality

Considering: $d_k < d_j$ $C_j = t + p_j$ $C'_j = t + p_k + p_j$
 $C_k = t + p_j + p_k$ $C'_k = t + p_k$

Then:

$$C'_j - d_j = C_k - d_j < C_k - d_k$$

$$C'_k - d_k = t + p_k - d_k < t + p_k + p_j - d_k = C_k - d_k$$

In general

$$\max_i C'_i - d'_i \leq \max_i C_i - d_i$$

Real-Time Short-Term Scheduling Algorithms

▪ Earliest Deadline First (EDF) – Earliest Due Date (EDD) – Optimality

Considering: $d_k < d_j$ $C_j = t + p_j$ $C'_j = t + p_k + p_j$
 $C_k = t + p_j + p_k$ $C'_k = t + p_k$

Then:

$$C'_j - d_j = C_k - d_j < C_k - d_k$$

$$C'_k - d_k = t + p_k - d_k < t + p_k + p_j - d_k = C_k - d_k$$

In general

$$\max_i C'_i - d'_i \leq \max_i C_i - d_i$$

→ switching i and j
– did not increase $\max_i C_i - d_i$
– makes S' closer to EDF

Real-Time Short-Term Scheduling Algorithms

▪ Earliest Deadline First (EDF) – Earliest Due Date (EDD) – Optimality

Considering: $d_k < d_j$ $C_j = t + p_j$ $C'_j = t + p_k + p_j$
 $C_k = t + p_j + p_k$ $C'_k = t + p_k$

Then:

$$C'_j - d_j = C_k - d_j < C_k - d_k$$

$$C'_k - d_k = t + p_k - d_k < t + p_k + p_j - d_k = C_k - d_k$$

In general

$$\max_i C'_i - d'_i \leq \max_i C_i - d_i$$

→ switching i and j
– did not increase $\max_i C_i - d_i$
– makes S' closer to EDF

=> Repeating the process for different j, k will lead to the EDF

Real-Time Short-Term Scheduling Algorithms

- **Earliest Deadline First (EDF) – Deadline Feasibility**
 - All processes are realizable if $L_{max} = \max_j (C_j - d_j) \leq 0$

Real-Time Short-Term Scheduling Algorithms

- **Earliest Deadline First (EDF) – Deadline Feasibility**
 - All processes are realizable if $L_{max} = \max_j (C_j - d_j) \leq 0$
 - Every deadline is feasible if and only if

$$u_i = \frac{1}{d_i} \sum_{k: d_k \leq d_i} p_k \leq 1$$

Real-Time Short-Term Scheduling Algorithms

- **Earliest Deadline First (EDF) – Deadline Feasibility**

- All processes are realizable if $L_{max} = \max_j (C_j - d_j) \leq 0$
- Every deadline is feasible if and only if

$$u_i = \frac{1}{d_i} \sum_{k:d_k \leq d_i} p_k \leq 1$$

with $u = \max u_i$ (load factor)

Real-Time Short-Term Scheduling Algorithms

- **Earliest Deadline First (EDF) – Deadline Feasibility**

- All processes are realizable if $L_{max} = \max_j (C_j - d_j) \leq 0$
- Every deadline is feasible if and only if

$$u_i = \frac{1}{d_i} \sum_{k: d_k \leq d_i} p_k \leq 1 \quad \text{with } u = \max u_i \text{ (load factor)}$$

- If $(u \leq 1) \rightarrow$ all deadlines are feasible.
- Elseif $(u > 1) \rightarrow$ minimize total of unmet deadlines $\sum U_j$

Real-Time Short-Term Scheduling Algorithms

- **Moore-Hodgson Algorithm (MHA)**
 - Same problem as EDF but aims at minimizing $\sum u_j$
 $\rightarrow 1 \mid \mid \sum U_j$

Real-Time Short-Term Scheduling Algorithms

- **Moore-Hodgson Algorithm (MHA)**
 - Same problem as EDF but aims at minimizing $\sum u_j$
 $\rightarrow 1 || \sum U_j$

Works in three steps

- STEP-1: Sort tasks J_i based on EDF order $\rightarrow [J_1, J_2, J_3, \dots J_n]$

Real-Time Short-Term Scheduling Algorithms

- **Moore-Hodgson Algorithm (MHA)**

- Same problem as EDF but aims at minimizing $\sum u_j$
→ $1 || \sum U_j$

Works in three steps

- STEP-1: Sort tasks J_i based on EDF order → $[J_1, J_2, J_3, \dots J_n]$
- STEP-2: Check whether there is a task that does not meet its deadline
 - If yes, go to STEP-3 ; Otherwise, stop.

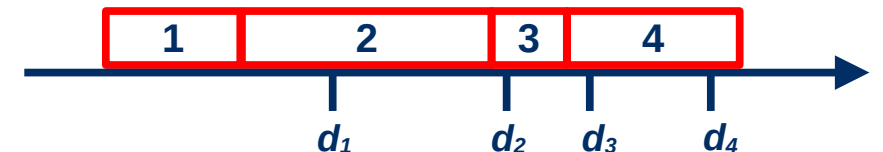
Real-Time Short-Term Scheduling Algorithms

■ Moore-Hodgson Algorithm (MHA)

- Same problem as EDF but aims at minimizing $\sum u_j$
→ $1 \mid \mid \sum U_j$

Works in three steps

- STEP-1: Sort tasks J_i based on EDF order $\rightarrow [J_1, J_2, J_3, \dots J_n]$
- STEP-2: Check whether there is a task that does not meet its deadline
 - If yes, go to STEP-3 ; Otherwise, stop.
- STEP-3: Let J_α be the first task that is late
 - remove task J_β with $p_\beta = \max_{j \leq \alpha} p_j$
 - Return to STEP-2



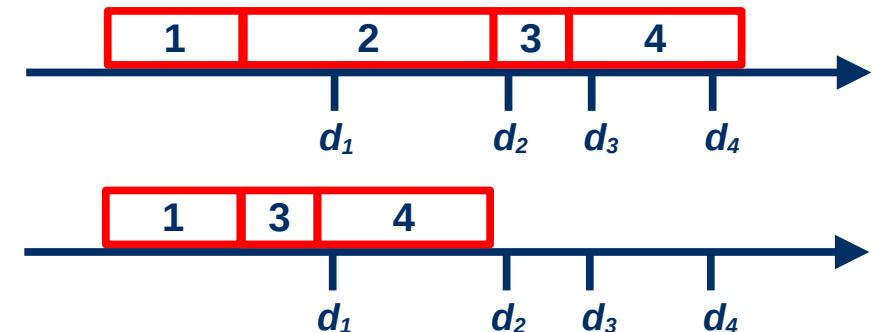
Real-Time Short-Term Scheduling Algorithms

■ Moore-Hodgson Algorithm (MHA)

- Same problem as EDF but aims at minimizing $\sum u_j$
→ $1 \mid \mid \sum U_j$

Works in three steps

- STEP-1: Sort tasks J_i based on EDF order $\rightarrow [J_1, J_2, J_3, \dots J_n]$
- STEP-2: Check whether there is a task that does not meet its deadline
 - If yes, go to STEP-3 ; Otherwise, stop.
- STEP-3: Let J_α be the first task that is late
 - remove task J_β with $p_\beta = \max_{j \leq \alpha} p_j$
 - Return to STEP-2



Real-Time Short-Term Scheduling Algorithms

+ Preconditions

Real-Time Short-Term Scheduling Algorithms

- **Preconditions: release/arrival-times**
 - In general $1|r_j|\Sigma U_j$ is a NP-hard problem.
 - EDF is optimal for $1|r_j, pmtn|\Sigma U_j$
 - Load $u = \sup_{t_1 < t_2 \leq \infty} u_{[t_1, t_2)}$ with

$$u_i = \frac{1}{d_i} \sum_{k: d_k \leq d_i} p_k \leq 1 \quad \longrightarrow \quad u_{[t_1, t_2)} = \frac{1}{t_2 - t_1} \sum_{j: t_1 \leq r_j, d_j \leq t_2} p_j$$

Real-Time Short-Term Scheduling Algorithms

- **Preconditions: release/arrival-times**

- In general $1|r_j|\Sigma U_j$ is a NP-hard problem.
- EDF is optimal for $1|r_j, pmtn|\Sigma U_j$
- Load $u = \sup_{t_1 < t_2 \leq \infty} u_{[t_1, t_2)}$ with

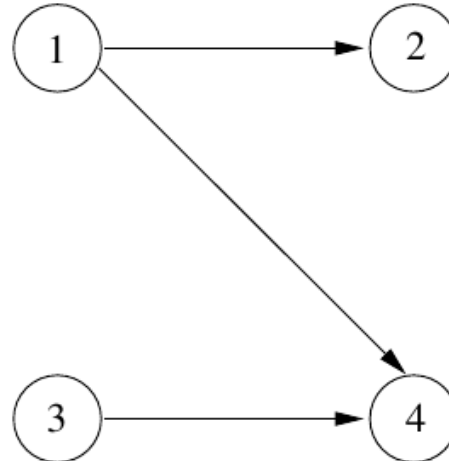
$$u_i = \frac{1}{d_i} \sum_{k: d_k \leq d_i} p_k \leq 1 \quad \longrightarrow \quad u_{[t_1, t_2)} = \frac{1}{t_2 - t_1} \sum_{j: t_1 \leq r_j, d_j \leq t_2} p_j$$

- Load $u \leq 1 \Leftrightarrow$ all deadlines feasible
- There are be some scenarios where EDF is optimal for $1|r_j|\Sigma U_j$
[no-idling scheduling orders]

Real-Time Short-Term Scheduling Algorithms

- **Preconditions: Precedence → Lawler's Algorithm**
 - Applicable when precedence preconditions are in place
[provided as a graph $G = (V,E)$]
 - Optimal for $1 | \text{prec} | L_{\max}$

Precedence graph



$p_1=1, p_2=2, p_3=2, p_4=1$
 $d_1=5, d_2=2, d_3=3, d_4=4$

Real-Time Short-Term Scheduling Algorithms

■ Preconditions: Precedence → Lawler's Algorithm

Two steps are involved:

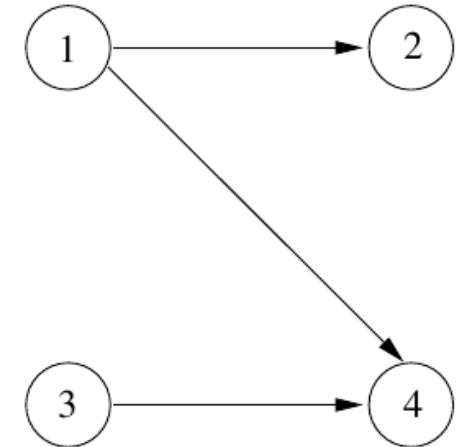
STEP-1: Assume

- $G = \text{precedence conditions}$, $num = |V|$
- $U(G) = \text{set of nodes of } G \text{ from which no edges depart}$
- $T = \sum p_j$, total execution time

STEP-2: choose task j in $U(G)$ with d_j is maximal (or $T - d_j$ minimal)

- Task j gets number num
- $num = num - 1$, $T = T - p_j$ and $G = G$ (remove node)
- Redefine $U(G)$ and repeat STEP-2

Precedence graph



$p_1=1, p_2=2, p_3=2, p_4=1$
 $d_1=5, d_2=2, d_3=3, d_4=4$

Real-Time Short-Term Scheduling Algorithms

▪ Scheduling Periodic Tasks

- Period of task i is T_i
- Each task has ∞ many instances with $r_j = jT_i$
- Instance has relative deadline T_i



Real-Time Short-Term Scheduling Algorithms

▪ Scheduling Periodic Tasks

- Period of task i is T_i
- Each task has ∞ many instances with $r_j = jT_i$
- Instance has relative deadline T_i



- EDF can handle n periodic tasks if and only if,
 - $U = \sum_i p_i/T_i \leq 1$ (with p_i/T_i = utilization of task i)
 - We prove this by showing that $u = U$
 - Thus, EDF is optimal for periodic tasks

Real-Time Short-Term Scheduling Algorithms

- Scheduling Periodic Tasks – EDF Optimality (proof $u = U$)

Considering:

$$u = \sup_{t_1 < t_2 \leq \infty} u[t_1, t_2) \quad u[t_1, t_2) = \frac{1}{t_2 - t_1} \sum_{j: t_1 \leq r_j, d_j \leq t_2} p_j \quad U = \sum_{j=1}^n p_j / T_j \leq 1.$$

Real-Time Short-Term Scheduling Algorithms

- Scheduling Periodic Tasks – EDF Optimality (proof $u = U$)

Considering:

$$u = \sup_{t_1 < t_2 \leq \infty} u_{[t_1, t_2)} \quad u_{[t_1, t_2)} = \frac{1}{t_2 - t_1} \sum_{j: t_1 \leq r_j, d_j \leq t_2} p_j \quad U = \sum_{j=1}^n p_j / T_j \leq 1.$$

Show that $u \leq U$ is for $t_1 < t_2$

$$u_{[t_1, t_2)}(t_2 - t_1) = \sum_{j: t_1 \leq r_j, d_j \leq t_2} p_j$$

Real-Time Short-Term Scheduling Algorithms

- Scheduling Periodic Tasks – EDF Optimality (proof $u = U$)

Considering:

$$u = \sup_{t_1 < t_2 \leq \infty} u_{[t_1, t_2)} \quad u_{[t_1, t_2)} = \frac{1}{t_2 - t_1} \sum_{j: t_1 \leq r_j, d_j \leq t_2} p_j \quad U = \sum_{j=1}^n p_j / T_j \leq 1.$$

Show that $u \leq U$ is for $t_1 < t_2$

$$u_{[t_1, t_2)}(t_2 - t_1) = \sum_{j: t_1 \leq r_j, d_j \leq t_2} p_j \leq \sum_{j=1}^n \left\lfloor \frac{t_2 - t_1}{T_j} \right\rfloor p_j$$

Real-Time Short-Term Scheduling Algorithms

- Scheduling Periodic Tasks – EDF Optimality (proof $u = U$)

Considering:

$$u = \sup_{t_1 < t_2 \leq \infty} u_{[t_1, t_2)} \quad u_{[t_1, t_2)} = \frac{1}{t_2 - t_1} \sum_{j: t_1 \leq r_j, d_j \leq t_2} p_j \quad U = \sum_{j=1}^n p_j / T_j \leq 1.$$

Show that $u \leq U$ is for $t_1 < t_2$

$$u_{[t_1, t_2)}(t_2 - t_1) = \sum_{j: t_1 \leq r_j, d_j \leq t_2} p_j \leq \sum_{j=1}^n \left\lfloor \frac{t_2 - t_1}{T_j} \right\rfloor p_j \leq (t_2 - t_1)U,$$

Real-Time Short-Term Scheduling Algorithms

- Scheduling Periodic Tasks – EDF Optimality (proof $u = U$)

Considering:

$$u = \sup_{t_1 < t_2 \leq \infty} u_{[t_1, t_2)} \quad u_{[t_1, t_2)} = \frac{1}{t_2 - t_1} \sum_{j: t_1 \leq r_j, d_j \leq t_2} p_j \quad U = \sum_{j=1}^n p_j / T_j \leq 1.$$

Show that $u \leq U$ is for $t_1 < t_2$

$$u_{[t_1, t_2)}(t_2 - t_1) = \sum_{j: t_1 \leq r_j, d_j \leq t_2} p_j \leq \sum_{j=1}^n \left\lfloor \frac{t_2 - t_1}{T_j} \right\rfloor p_j \leq (t_2 - t_1)U,$$

Show that $u \geq U$ by replacing $t_1 = 0$ and $t_2 = H$ [smallest number divisible by T_j]

$$u_{[0, H)}H = \sum_{j: 0 \leq r_j, d_j \leq H} p_j = \sum_{j=1}^n \frac{H}{T_j} p_j = HU.$$

Real-Time Short-Term Scheduling Algorithms

- Scheduling Periodic Tasks – EDF Optimality (proof $u = U$)

Considering:

$$u = \sup_{t_1 < t_2 \leq \infty} u_{[t_1, t_2)} \quad u_{[t_1, t_2)} = \frac{1}{t_2 - t_1} \sum_{j: t_1 \leq r_j, d_j \leq t_2} p_j \quad U = \sum_{j=1}^n p_j / T_j \leq 1.$$

Show that $u \leq U$ is for $t_1 < t_2$

$$u_{[t_1, t_2)}(t_2 - t_1) = \sum_{j: t_1 \leq r_j, d_j \leq t_2} p_j \leq \sum_{j=1}^n \left\lfloor \frac{t_2 - t_1}{T_j} \right\rfloor p_j \leq (t_2 - t_1)U,$$

Show that $u \geq U$ by replacing $t_1 = 0$ and $t_2 = H$ [smallest number divisible by T_j]

$$u_{[0, H)} H = \sum_{j: 0 \leq r_j, d_j \leq H} p_j = \sum_{j=1}^n \frac{H}{T_j} p_j = HU.$$

Since u is the supremum
 $\rightarrow u \geq U$

Real-Time Short-Term Scheduling Algorithms

- **Rate Monotonic Scheduling (RMS)**

- Combining periodic real-time tasks with non-real-time tasks
- Handle classical UNIX scheduler + added priorities.

[real-time has highest priority and each real time task has a fixed priority]

Real-Time Short-Term Scheduling Algorithms

- **Rate Monotonic Scheduling (RMS)**

- Combining periodic real-time tasks with non-real-time tasks
- Handle classical UNIX scheduler + added priorities.

[real-time has highest priority and each real time task has a fixed priority]

- **Liu and Layland proposal:**

- Organize tasks so that $T_1 \leq T_2 \leq \dots \leq T_n$,
- Returns task with period T_i priority i (highest is 1)

Real-Time Short-Term Scheduling Algorithms

- **Rate Monotonic Scheduling (RMS) – deadline feasibility**

- RMS respects all deadlines certainly if

$$U = \sum_i p_i/T_i \leq n(2^{1/n}-1)$$

If $n \rightarrow \infty$ then $n(2^{1/n}-1)$ drops to $\ln 2 = 0.693$

- RMS is optimal within class of static priority-driven systems
[relative deadlines are equal to the period]
- RMS will meet all deadlines If all periods are multiples of each other then and $U \leq 1$

Key points

Processor Scheduling

Pay attention to...

- Earliest Deadline First, when is it optimal? when are the deadlines feasible?
- Operation Moore-Hodgson, what is minimized?
- How to take precedence into account (Lawler algorithm)?
- Rate-monotonic scheduling, What is its Purpose? What it guarantees?

Further Reading

Essential: Scheduling

- OS Course Notes - chapter 5.
- Blackboard – Recorded lectures from Prof. Van Houdt.
 - Scheduling deel III and IV

Questions?



Operating Systems

[1500WETOPS]

José Oramas